EXPERIMENT 7:

Write a C program to find FIRST( ) - predictive parser for the given grammar

S → AaAb / BbBa

A → ∈

B → ∈

CODE:

```
#include <stdio.h>

int main() {
   printf("FIRST(A) = { epsilon }\n");
   printf("FIRST(B) = { epsilon }\n");
   printf("FIRST(S) = { a, b }\n");
   return 0;
}
```

OUTPUT:

Output

```
FIRST(A) = { epsilon }
FIRST(B) = { epsilon }
FIRST(S) = { a, b }
```

EXPERIMENT 8:

Write a C program to find FOLLOW( ) - predictive parser for the given grammar

S → AaAb / BbBa

A → ∈

B → ∈

CODE:

```
#include <stdio.h>

int main() {
   printf("FOLLOW(S) = { $ }\n");
```

```
    printf("FOLLOW(A) = { a, b }\n");
    printf("FOLLOW(B) = { a, b }\n");
    return 0;
}
```

OUTPUT:



EXPRIMENT 9:

Implement a C program to eliminate left recursion from a given CFG.

S → (L) / a

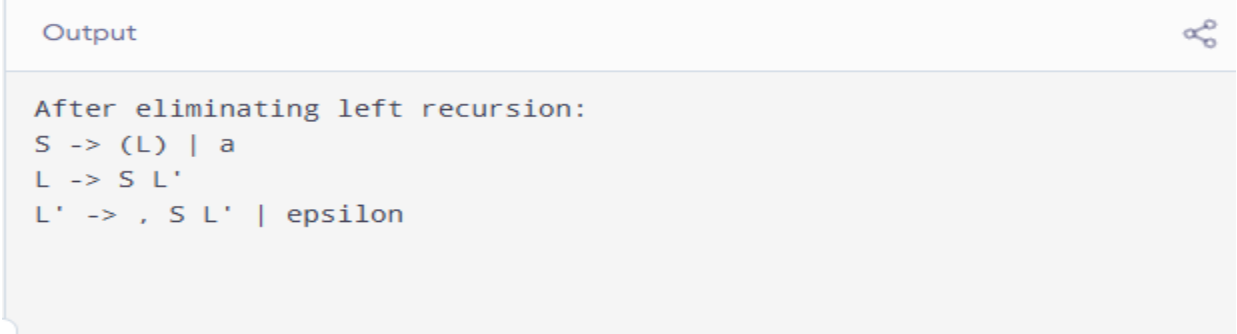
L → L , S / S

CODE:

```
#include <stdio.h>

int main() {
    printf("After eliminating left recursion:\n");
    printf("S -> (L) | a\n");
    printf("L -> S L'\n");
    printf("L' -> , S L' | epsilon\n");
    return 0;
}
```

OUTPUT:



EXPERIMENT 10:
Implement a C program to eliminate left factoring from a given CFG.
S → iEtS / iEtSeS / a
E → b
CODE:

```
#include <stdio.h>

int main() {
   printf("After eliminating left factoring:\n");
   printf("S -> iEtSS' | a\n");
   printf("S' -> eS | epsilon\n");
   printf("E -> b\n");
   return 0;
}
```

OUTPUT:

Output

```
After eliminating left factoring:
S -> iEtSS' | a
S' -> eS | epsilon
E -> b
```

EXPERIMENT 11:
Implement a C program to perform symbol table operations.
CODE:

```
#include <stdio.h>
#include <string.h>

struct Symbol {
   char name[20];
   int value;
} s[10];

int main() {
```

```
    int n = 0, ch, i;
    char name[20];

    while (1) {
        printf("\n1.Insert 2.Search 3.Display 4.Exit\n");
        scanf("%d", &ch);

        if (ch == 1) {
            printf("Enter name and value: ");
            scanf("%s%d", s[n].name, &s[n].value);
            n++;
        }
        else if (ch == 2) {
            printf("Enter name: ");
            scanf("%s", name);
            for (i = 0; i < n; i++)
                if (strcmp(s[i].name, name) == 0)
                    break;
            if (i < n) printf("Found: %d\n", s[i].value);
            else printf("Not found\n");
        }
        else if (ch == 3) {
            for (i = 0; i < n; i++)
                printf("%s %d\n", s[i].name, s[i].value);
        }
        else break;
    }
    return 0;
}
```

OUTPUT:

Output

```
1.Insert 2.Search 3.Display 4.Exit
1
x 10
2
x
3
4
Enter name and value:
1.Insert 2.Search 3.Display 4.Exit
Enter name: Found: 10

1.Insert 2.Search 3.Display 4.Exit
x 10

1.Insert 2.Search 3.Display 4.Exit
```